

[Click here to view the PDF version.](#)

Error message from Main

The package `Plots.jl` could not load because it failed to initialize.

That's not nice! Things you could try:

- Restart the notebook.
- Try a different Julia version.
- Contact the developers of `Plots.jl` about this error.

You might find useful information in the package installation log:

```
@ Base ./loading.jl:2271
[19] #invoke_in_world#3
@ ./essentials.jl:1089 [inlined]
[20] invoke_in_world
@ ./essentials.jl:1086 [inlined]
[21] require(into::Module, mod::Symbol)
@ Base ./loading.jl:2260
[22] include
@ ./Base.jl:557 [inlined]
[23] include_package_for_output(pkg::Base.PkgId, input::String, depot_path::Vector{String}, d
l_load_path::Vector{String}, load_path::Vector{String}, concrete_deps::Vector{Pair{Base.PkgId,
UInt128}}, source::Nothing)
@ Base ./loading.jl:2881
[24] top-level scope
@ stdin:6
```

```
1 begin
2     using Plots
3     using PlutoUI
4     using PlutoTeachingTools
5     using LaTeXStrings
6     using DifferentialEquations
7     using ForwardDiff
8     using HypertextLiteral: @html, @html_str
9 end
```

Error message

UndefVarError: ``TableOfContents`` not defined in ``Main.var"workspace#3"``

Suggestion: check for spelling errors or missing imports.

Stack trace

Here is what happened, the most recent locations are first:

1. from `This cell: line 1`

1 `TableOfContents()`

1 `TableOfContents()`

 *Reading hidden code*

Initial value problems

Error message

```
UndefVarError: `TODO` not defined in `Main.var"workspace#3"`  
Suggestion: check for spelling errors or missing imports.
```

Stack trace

Here is what happened, the most recent locations are first:

1. from | This cell: line 1

```
1 TODO(md"Start with example and introduce general definition thereafter  
r.")
```

```
1 TODO(md"Start with example and introduce general definition thereafter.")
```

In computational science we are often faced with quantities that change continuously in space or time. For example the temperature profile of a hot body or the population of an animal species over time. Such problems are often modelled by differential equations. If there is only a single independent variable t we call the model an **ordinary differential equation**. The usual setup is that for initial value $t = 0$ we have some knowledge about our problem, e.g. by performing a measurement, and the key question is thus how the situation evolves for $t > 0$. Mathematically we call this an **initial value problem**, for example:

Definition: Initial-value problem

A scalar, first-order initial value problem (IVP) can be formulated as

$$\begin{cases} \frac{du(t)}{dt} = f(t, u(t)) & a \leq t \leq b \\ u(a) = u_0, \end{cases} \quad (1)$$

We call t the **independent variable**, u the **dependent variable** and u_0 the initial conditions.

A **solution** of an initial-value problem is a differentiable, continuous function $u : \mathbb{R} \rightarrow \mathbb{R}$ which makes both $u'(t) = f(t, u(t))$ (for all $a \leq t \leq b$) and $u(a) = u_0$ true equations.

For the specific case where $f(t, u) = g(t) + uh(t)$ for two functions g and h the differential equation (1) is called **linear**.

Often (but not always) t plays the role of time and (1) thus models the time-dependence of a quantity u . Let us consider an example:

Example: Ducks on the Leman

Suppose we want to model the population of ducks on the Leman, i.e. let $u(t)$ denote the number of ducks on the lake at time t . To simplify the modelling we allow for "fractional ducks", i.e. we allow u to be any real number.

First we assume a constant growth rate C for the number of ducks at any time t , i.e. that the number of ducks born minus the number of ducks deceased in one unit of time is proportional to $u(t)$ — the current population of ducks. This leads to the ordinary differential equation (ODE)

$$\begin{cases} \frac{du(t)}{dt} = C u(t) & t > 0 \\ u(0) = u_0 \end{cases}$$

where u_0 is the initial number of ducks we observed at $t = 0$. In comparison to the general definition shown above we thus have $f(t, u) = C u$.

The solution of this linear equation is

$$u(t) = u_0 e^{Ct},$$

i.e. exponential growth.

Clearly as the size of the Leman is finite, this is not a reasonable model as both food and space on the lake is limited, which should cap the growth. To improve the model we assume that the death rate itself is proportional to the size of the population, i.e. du for a constant $d > 0$. Keeping the idea of a constant birth rate $b > 0$ one thus obtains an improved model

$$\begin{cases} \frac{du(t)}{dt} = b u(t) - d (u(t))^2 & t > 0 \\ u(0) = u_0 \end{cases}$$

i.e. we have the case of $f(u, t) = b u - d u^2$ when considering the above definition.

This is the **logistic equation**, which in fact has multiple solutions. The solution relevant for population models has the form

$$u(t) = \frac{b/d}{1 + \left(\frac{b}{d u_0} - 1\right) e^{-bt}}$$

As a plot shows, this solution varies smoothly from some initial population u_0 to a final population b/d :

Error message

```
UndefVarError: `@L_str` not defined in `Main.var"workspace#3"`  
Suggestion: check for spelling errors or missing imports.
```

Stack trace

Here is what happened, the most recent locations are first:

1. from [:0
2. #macroexpand#74
from [expr.jl:125
3. macroexpand
from [expr.jl:123

```
1 let  
2     u0 = 1  
3     b = 4  
4     d = 0.5  
5     u(t) = b/d / (1 + (b/(d*u0) - 1) * exp(-b*t))  
6  
7     plot(u; xlims=(0, 3), label="b=4, d=0.5, u0=1", xlabel=L"t", ylabel=L"u",  
8         title="Duck population", lw=2)  
8 end
```

Error message

```
UndefVarError: `TODO` not defined in `Main.var"workspace#3"`  
Suggestion: check for spelling errors or missing imports.
```

Stack trace

Here is what happened, the most recent locations are first:

1. from **This cell: line 1**

```
1 TODO(md"Avoid introducing DifferentialEquations.jl and the ODEProblem o  
bject.")
```

```
1 TODO(md"Avoid introducing DifferentialEquations.jl and the ODEProblem object.")
```

Numerical solution in DifferentialEquations.jl

For such simple examples analytic solutions can still be found with a little practice. However, for more involved cases analytical solutions can be more tricky or impossible and we will discuss numerical techniques to solve such equations. Before we discuss a few simple methods in detail, we first consider the [DifferentialEquations.jl](#) Julia package, which provides a range of production-grade numerical solvers for ODE problems and will serve us as the reference method in our discussion.

As an example we consider the more involved initial value problem

$$\begin{cases} \frac{du(t)}{dt} = \sin[(u+t)^2] & 0 \leq t \leq 4 \\ u(0) = -1, \end{cases} \quad (2)$$

for which an analytical solution is not so easy to obtain. To solve it using DifferentialEquations we first construct an ODEProblem object, which is essentially a Julia representation of (2):

Error message

```
UndefVarError: `ODEProblem` not defined in `Main.var"workspace#3"`  
Suggestion: check for spelling errors or missing imports.
```

Stack trace

Here is what happened, the most recent locations are first:

```
1. from | This cell: line 5  
4 tspan = (0.0, 4.0) # t interval  
5 ivp = ODEProblem(f, u0, tspan)  
6 end
```

```
1 begin  
2     f = (u, p, t) -> sin((t + u)^2) # defines du/dt, must include p argument  
3     u0 = -1.0 # initial value  
4     tspan = (0.0, 4.0) # t interval  
5     ivp = ODEProblem(f, u0, tspan)  
6 end
```

A good default solver for such problems is the `Tsit5` method, which can be employed as follows:

Error message

```
UndefVarError: `solve` not defined in `Main.var"workspace#3"`  
Suggestion: check for spelling errors or missing imports.
```

```
1 sol = solve(ivp, Tsit5());
```

The obtained solution can easily be visualised:

Error message

```
UndefVarError: `@L_str` not defined in `Main.var"workspace#3"`  
Suggestion: check for spelling errors or missing imports.
```

```
1 plot(sol, label="solution", lw=2, xlabel="t", ylabel=L"u(t)",  
2      title=L"\frac{du}{dt} = \sin((t+u)^2)")
```

Optional: Existence and uniqueness of solutions

Before we discuss some numerical techniques for solving initial value problems in detail, a quick reminder that in general such an IVP can by itself not be solvable or admit more than one solution.

First consider the case where the **existence** of a solution to problem (1) is not guaranteed for all t with $a \leq t \leq b$. For example consider the problem

$$\frac{du(t)}{dt} = (u(t))^2, \quad 0 \leq t \leq 2, \quad u(0) = 1,$$

which has solution $u(t) = \frac{1}{1-t}$. This solution only exists for $t < 1$. When attempting a numerical solution beyond $t = 1$ of such a problem we are faced with problems:

Error message

```
UndefVarError: `@L_str` not defined in `Main.var"workspace#3"`  
Suggestion: check for spelling errors or missing imports.
```

Stack trace

Here is what happened, the most recent locations are first:

1. from [:0
2. #macroexpand#74
from [expr.jl:125
3. macroexpand
from [expr.jl:123

```
1 let  
2   f = (u, p, t) -> u^2  
3   u_0 = 1.0  
4   tspan = (0, 10)  
5  
6   ivp = ODEProblem(f, u_0, tspan)  
7   sol = solve(ivp, Tsit5())  
8  
9   plot(sol; lw=2, label="", xlabel=L"t", yaxis=:log, ylabel=L"u(t)", xlims=(0,  
10  2))  
10 end
```

A second issue is that the solution may not necessarily be **unique**. Consider for example the equation

$$\frac{du(t)}{dt} = \sqrt{u(t)}, \quad t > 0, \quad u(0) = 0,$$

which has the two solutions $u_1(t) = 0$ and $u_2(t) = \frac{1}{4}t^2$.

Both cases cause additional difficulties to numerical techniques, which we do not want to discuss here. Instead we thus recall the following result ensuring the existence of a unique solution to (1):

Theorem 1: Existence and uniqueness of first-order ODEs

Given $f : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$ a continuous function with respect to both arguments and **Lipschitz-continuous** with respect to its second argument, i.e. there exists a constant $L > 0$ such that

$$|f(t, x) - f(t, y)| \leq L|x - y|, \quad \forall x, y \in \mathbb{R}, \forall t \text{ with } a \leq t \leq b$$

Then the ODE problem (1) has a unique solution $u(t)$ defined for all $a \leq t \leq b$ and u is moreover continuously differentiable.

For the rest of this chapter we will always assume this condition to be satisfied.

Forward Euler

Error message

```
UndefVarError: `TODO` not defined in `Main.var"workspace#3"`  
Suggestion: check for spelling errors or missing imports.
```

Stack trace

Here is what happened, the most recent locations are first:

1. from `This cell: line 1`

```
1 TODO(md"Mention the general idea of solving the problem: Knowing an approximation to the solution at some time $t_n$ we want to understand how we can find the solution at some later time $t_{n+1}$.")
```

```
1 TODO(md"Mention the general idea of solving the problem: Knowing an approximation to the solution at some time $t_n$ we want to understand how we can find the solution at some later time $t_{n+1}$.")
```

Let's assume we want to solve the IVP (1) in the interval $[a, b]$, i.e.

$$\begin{cases} \frac{du(t)}{dt} = f(t, u(t)) & a \leq t \leq b \\ u(a) = u_0, \end{cases} \quad (3)$$

A natural idea is to divide this interval into N subintervals $[t_n, t_{n+1}]$ of uniform length h , i.e.

$$t_n = a + n h \quad n = 0, \dots, N \quad h = \frac{b - a}{N}, \quad (4)$$

where h is also called the **stepsize**. At time t_n we thus need to satisfy

$$\frac{d u(t_n)}{dt} = f(t_n, u(t_n)),$$

where both $u(t_n)$ are unknown as well as the derivative on the LHS are unknown. To reduce this to a single unknown we replace the derivative by one of the finite differences formulas discussed in the previous chapter on numerical differentiation.

The simplest approach is the **forward Euler method** in which one approximates $\frac{d u(t_n)}{dt}$ by forward finite differences, i.e.

$$\frac{d u(t_n)}{dt} \approx D_h^+ u(t_n) = \frac{1}{h} (u(t_n + h) - u(t_n)) = \frac{1}{h} (u(t_{n+1}) - u(t_n)). \quad (5)$$

If by $u^{(n)}$ we denote an approximation to $u(t_n)$ and similarly for $u(t_{n+1})$ we obtain from (3):

$$\begin{cases} \frac{u^{(n+1)} - u^{(n)}}{h} = f(t_n, u^{(n)}), & \forall n = 0, \dots, N-1 \\ u^{(0)} = u(a) = u_0 \end{cases} \quad (6)$$

This defines a numerical scheme:

Algorithm 1: Forward Euler method

Given an initial value problem $\frac{du}{dt} = f(u, t)$, $u(0) = u_0$ with $t \in [a, b]$ and nodes $t_n = a + n h$, $h = \frac{b-a}{N}$ iteratively compute the sequence

$$u^{(n+1)} = u^{(n)} + h f(t_n, u^{(n)}) \quad \forall n = 0, \dots, N-1. \quad (7)$$

Then $u^{(n)}$ is *approximately* the solution u at $t = t_n$.

Notice, that the replacement of the derivative $\frac{d u(t_n)}{dt}$ in (6) by the forward finite difference approximation implies that the computed points $(u^{(0)}, u^{(1)}, \dots, u^{(N)})$ are **not equal to** the function values of the exact solution u .

A basic implementation of Euler's method is shown below. It expects the number of time intervals N as well as the IVP — in form of an `ODEProblem`. All other problem parameters (a, b, f

, ...) are then extract from the latter object. The output is the vector of nodes t_n and the vector of approximate solution values $u^{(n)} \approx u(t_n)$.

forward_euler (generic function with 1 method)

```
1 function forward_euler(ivp, N)
2     a, b = ivp.tspan # Extract time interval [a, b]
3     u0 = ivp.u0      # Initial condition
4     f = ivp.f        # Derivative function f
5     p = ivp.p        # Further parameters (not used in our implementation)
6
7     # Discrete time nodes to consider:
8     h = (b - a) / N
9     t = [a + i * h for i in 0:N]
10
11     # Setup output vector with all elements initialised to u0
12     u = [copy(u0) for i in 0:N]
13
14     # Time integration
15     u[1] = u0
16     for i in 1:N
17         u[i+1] = u[i] + h * f(u[i], p, t[i])
18     end
19
20     (; t, u)
21 end
```

Let us compare this approach to the test problem (2)

$$\begin{cases} \frac{du(t)}{dt} = \sin[(u+t)^2] & 0 \leq t \leq 4 \\ u(a) = 1, \end{cases}$$

which we solved previously using DifferentialEquations. The ivp object already contains a setup of this initial value problem

Error message

Another cell defining `ivp` contains errors.

```
1 ivp
```

and we can directly proceed to solving it:

Error message

Another cell defining `ivp` and `Neuler` contains errors.

```
1 res_euler = forward_euler(ivp, Neuler);
```

and plot the result:

Error message

UndefVarError: `@L\_str` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.

```
1 let
2   res_euler = forward_euler(ivp, Neuler)
3
4   plot(sol, label="reference", lw=2, xlabel="t", ylabel=L"u(t)",
5         title=L"\frac{du}{dt} = \sin((t+u)^2)", ylims=(-2, 0))
6
7   plot!(res_euler.t, res_euler.u; label="Euler N=$Neuler", mark=:o, lw=2,
8         ls=:dash)
9 end
```

👁 Reading hidden

Error message

UndefVarError: `Slider` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.

Stack trace

Here is what happened, the most recent locations are first:

1. macro expansion
from `bonds.jl:125`

2. from `This cell: line 1`

```
1 md"- Number of subintervals: `Neuler = ` $(@bind Neuler Slider(5:1:50;
show_value=true, default=20)) "
```

```
1 md"- Number of subintervals: `Neuler = ` $(@bind Neuler Slider(5:1:50;
show_value=true, default=20)) "
```

👁 Reading hidden

We observe that the approximated solution becomes more and more accurate as we increase the number of sub-intervals N and as the step size h decreases.

But we also observe that for too small values of N (e.g. $N = 9$) that the Euler solution notably deviates from the reference.

Forward Euler is just one example of a broader class of numerical algorithms for initial value problems:

Definition: One-step methods for initial value problems

A **one-step method** to solve (1) is a method of the form

$$u^{(n+1)} = u^{(n)} + h \phi_f(u^{(n)}, t_n, h) \quad \text{for } n = 0, 1, \dots, N-1, \quad (8)$$

where $\phi_f(u, t, h)$ is a real-valued function and $u^{(0)} = u_0$.

Examples

- For **Forward Euler** we have $\phi_f(u^{(n)}, t_n, h) = f(t_n, u^{(n)})$
- More involved methods exist. For example the **midpoint method**, which we will discuss below, employs $\phi_f(u^{(n)}, t_n, h) = f\left(t_n + \frac{h}{2}, u^{(n)} + \frac{h}{2} f(t_n, u^{(n)})\right)$.

Error analysis

As discussed above applying Forward Euler to an initial value problem (1) with N subintervals leads to a sequence of computed points $u^{(0)}, u^{(1)}, \dots, u^{(N)}$, which approximate $u(t_0), u(t_1), \dots, u(t_N)$ — the solution u evaluated at the nodes $\{t_i\}_{i=0}^N$. In this section our goal is to quantify the error

$$|e^{(n)}| = |u(t_n) - u^{(n)}| \quad \text{for } n = 0, 1, \dots, N \quad (9)$$

between the computed points and the exact values of the solution.

A notable feature of Forward Euler (Algorithm 1) is that the expression for computing $\mathbf{u}^{(n+1)}$ results from two approximations:

1. Instead of evaluating the exact derivative $\frac{d\mathbf{u}(t_n)}{dt}$ we employ a finite differences formula in (5).
2. Instead of using the exact value $\mathbf{u}(t_n)$ to compute $\mathbf{u}^{(n+1)}$ in (6) we employ the *approximation* $\mathbf{u}^{(n)}$.

Both approximations contribute to the total error (9).

Local truncation error

We first want to understand the error due to employing the finite difference formula instead of the exact derivative. To isolate this contribution let us assume that we do have access to the *exact* values $\mathbf{u}(t_n)$. We then follow a Forward Euler step (7), just replacing the approximation $\mathbf{u}^{(n)}$ by the exact $\mathbf{u}(t_n)$. This leads to the quantity

$$\tilde{\mathbf{u}}^{(n+1)} = \mathbf{u}(t_n) + h\mathbf{f}(t_n, \mathbf{u}(t_n)),$$

which thus isolates the error due to employing the finite difference formula. We investigate the error of this quantity using a Taylor series

$$\begin{aligned} \mathbf{u}(t_{n+1}) - \tilde{\mathbf{u}}^{(n+1)} &= \mathbf{u}(t_n + h) - \left[\mathbf{u}(t_n) + h \underbrace{\mathbf{f}(t_n, \mathbf{u}(t_n))}_{=\mathbf{u}'(t_n)} \right] \\ &= \mathbf{u}(t_n) + h\mathbf{u}'(t_n) + \frac{1}{2}h^2\mathbf{u}''(t_n) + O(h^3) - \mathbf{u}(t_n) - h\mathbf{u}'(t_n) \\ &= \frac{1}{2}h^2\mathbf{u}''(t_n) + O(h^3), \end{aligned}$$

where we used the fact that $\mathbf{u}$ satisfies the differential equation. This expression only describes the error of a single step, which advances us by a stepsize h in going from $\mathbf{a}$ to $\mathbf{b}$. To make the scaling of errors comparable as $h \rightarrow 0$ it is usually more convenient to investigate its size relative to the chosen stepsize h , i.e.

$$\frac{\mathbf{u}(t_{n+1}) - \tilde{\mathbf{u}}^{(n+1)}}{h} = \frac{\mathbf{u}(t_{n+1}) - \mathbf{u}(t_n)}{h} - \mathbf{f}(t_n, \mathbf{u}(t_n)) = \frac{1}{2}h\mathbf{u}''(t_n) + O(h^2). \quad (10)$$

This last expression is the so-called **local truncation error** of forward Euler.

Generalising towards the general class of one-step methods we define:

Definition: Local truncation error

The **local truncation error** of a one-step method (8) is the quantity

$$\tau_h^{(n)} = \frac{u(t_{n+1}) - u(t_n)}{h} - \phi_f(u(t_n), t_n, h) \quad (11)$$

There are two ways to interpret the local truncation error $\tau_h^{(n)}$:

- As we discussed above (10) with respect to the Forward Euler method, it measures the **error of the numerical scheme** going from step n to $n + 1$, provided that we start from the exact $u(t_n)$.
- From definition (9) we can also view $\tau_h^{(n)}$ as the (relative) **residual of the numerical method** when replacing the numerical solution $u^{(n)}$ by the exact solution $u(t_n)$.

Global error

As the name suggests the local truncation error $\tau^{(n)}$ only makes a statement about the error committed in a single isolated time step. However, what we were after was estimating the *global* error

$$|e^{(n)}| = |u(t_n) - u^{(n)}| \quad \text{for } n = 0, 1, \dots, N.$$

Naively one might think that simply adding all local error contributions provides an estimate for this global error. However, this neglects something important. Namely, due to the algorithmic structure of one-step methods (8), where one obtains $u^{(n+1)}$ by adding to the approximation $u^{(n)}$, the error of step n *propagates* to step $n + 1$. In particular for small values of N this can cause the error $|u(t_n) - u^{(n)}|$ in later time steps to grow significantly over the earlier ones. For example consider:

Error message

```
UndefinedVarError: '@L_str' not defined in 'Main.var"workspace#3"'
Suggestion: check for spelling errors or missing imports.
```

```
1 let
2   res_euler = forward_euler(ivp, 7)
3   plot(sol, label="reference", lw=2, xlabel="t", ylabel=L"u(t)",
```



```

4     title=L"\frac{du}{dt} = \sin((t+u)^2)", ylims=(-2, 1))
5     plot!(res_euler.t, res_euler.u; label=L"Euler $N=7$", mark=:o, lw=2,
6         ls=:dash)
6 end

```

👁 Reading hidden

where from time step 5 the solution is qualitatively wrong.

We therefore need to account for the propagation of error from one step to the next on top of the local error measured by $\tau_h^{(n)}$. This is the subject of the following theorem:

Theorem 2

Given a one-step method (8) with local truncation error satisfying

$$|\tau_h^{(n)}| \leq C h^p \quad (12)$$

with a constant $C > 0$ and $p \in \mathbb{N}$ as well as [1]

$$\frac{\partial \phi_f(u, t, h)}{\partial u} \leq L \quad \text{for all } t \in [a, b] \text{ and all } h > 0. \quad (13)$$

Then the global error satisfies for all $n = 0, 1, \dots, N$

$$|e^{(n)}| = |u(t_n) - u^{(n)}| \leq \frac{Ch^p}{L} (e^{L(t_n-a)} - 1) \quad (14)$$

as $h \rightarrow 0$, that is the numerical method (8) is **convergent with order p** .

Error message

```

UndefVarError: `details` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.

```

Stack trace

Here is what happened, the most recent locations are first:

1. from `This cell: line 1`

```

1 details("Show the proof",
2 md"""

```

```

1 details("Show the proof",
2 md"""
3 > **Proof:**
4 > Based on  $e^{(n)} = u(t_n) - u^{(n)}$  we obtain
5 > ```math
6 > \begin{aligned}
7 > e^{(n+1)} - e^{(n)} &= [u(t_{n+1}) - u^{(n+1)}] - [u(t_n) - u^{(n)}] \\
8 > &= [u(t_{n+1}) - u(t_n)] - [u^{(n+1)} - u^{(n)}] \\
9 > &\stackrel{(8)}{=} [u(t_{n+1}) - u(t_n)] - h\,, \phi_f(u^{(n)}, t_n, h).
10 > \end{aligned}
11 > ```
12 > By adding and subtracting  $h\,, \phi_f(u(t_n), t_n, h)$  we develop this
    to
13 > ```math
14 > \begin{aligned}
15 > e^{(n+1)} &= e^{(n)} + \overbrace{[u(t_{n+1}) - u(t_n) - h\,, \phi_f(u(t_n), t_n, h)]}^{=h\,, \tau^{(n)}_h} \\
16 > &+ h [\phi_f(u(t_n), t_n, h) - \phi_f(u^{(n)}, t_n, h)] \\
17 > \end{aligned}
18 > ```
19 > Employing the triangle inequality yields
20 > ```math
21 > |e^{(n+1)}| \leq |e^{(n)}| + h |\tau^{(n)}_h| + h \left| \phi_f(u(t_n), t_n, h) - \phi_f(u^{(n)}, t_n, h) \right|
22 > ```
23 > Concerning the last term, the Fundamental Theorem of Calculus implies
24 > ```math
25 > \begin{aligned}
26 > \left| \phi_f(u(t_n), t_n, h) - \phi_f(u^{(n)}, t_n, h) \right| \\
27 > &= \left| \int_{u^{(n)}}^{u(t_n)} \frac{\partial \phi_f(u, t_n, h)}{\partial u} du \right| \\
28 > &\leq \int_{u^{(n)}}^{u(t_n)} \left| \frac{\partial \phi_f(u, t_n, h)}{\partial u} \right| du \\
29 > &\stackrel{(13)}{\leq} L \left| u(t_n) - u^{(n)} \right| = L |e^{(n)}|.
30 > \end{aligned}
31 > ```
32 > Therefore using (12)
33 > ```math
34 > \begin{aligned}
35 > |e^{(n+1)}| &\leq (1 + hL) \left| e^{(n)} \right| + h |\tau^{(n)}_h| \\
36 > &\stackrel{(12)}{\leq} C\,, h^{p+1} + (1 + hL) \left| e^{(n)} \right|
37 > \end{aligned}
38 > with which we develop
39 > ```math
40 > \begin{aligned}
41 > |e^{(n+1)}| &\leq C\,, h^{p+1} + (1 + hL) \left| e^{(n)} \right|
42

```

```

43 > &\leq C\,h^{p+1} + (1 + h\,L) \left( C\,h^{p+1} + (1 + h\,L)\, \left| e^{\{(n-1)\}} \right| \right) \\
44 > &= C\,h^{p+1} + (1 + h\,L) \, C\,h^{p+1} + (1+h\,L)^2\, \left| e^{\{(n-1)\}} \right| \\
45 & \\
46 > &\vdots \\
47 > &\sum_{i=0}^n (1+h\,L)^i\, C\,h^{p+1} + (1+h\,L)^{n+1} |e_0| \\
48 > &= \frac{1-(1+h\,L)^{n+1}}{1 - (1+h\,L)} C\,h^{p+1} + (1+h\,L)^{n+1} |e_0|. \\
49 > \end{aligned} \\
50 > \\
51 > \text{where in the last step we used the identity} \\
52 > \text{``math} \\
53 > \sum_{i=0}^n r^i = \frac{1-r^{n+1}}{1-r} \quad \text{for all } r \neq 1 \\
54 > \text{Now we notice that } e_0 = u(a) - u_0 = 0, \text{ such that the expression simplifies} \\
55 \text{ to} \\
56 > \text{``math} \\
57 > |e^{\{(n+1)\}}| \leq \frac{1-\left(1+h\,L\right)^{n+1}}{-h\,L} C\,h^{p+1} \\
58 > = \frac{C\,h^p}{L} \left( \left(1+h\,L\right)^{n+1} - 1 \right) \\
59 > \\
60 > \text{Finally notice that } 1 + x \leq e^x \text{ for all } x \geq 0 \text{ and therefore that} \\
61 > \left(1+h\,L\right)^{n+1} \leq e^{\{(n+1)\}h\,L} = e^{\{(t_{n+1}-a)\}L}. \text{ Replacing } n+1 \text{ by } n \\
62 > \text{we obtain} \\
63 > \text{``math} \\
64 > |e^{\{(n)\}}| \leq \frac{C\,h^p}{L} \left( e^{\{(t_n-a)\}L} - 1 \right) \\
65 > \\
66 > \text{which concludes the proof.}

```

 Reading hidden

[1]:

This result can also be proven with minor modifications under the conditions of Theorem 1, i.e. that $|\phi_f(u, t, h) - \phi_f(v, t, h)| \leq L|u - v|$ for all $u, v \in \mathbb{R}$, all $h > 0$ and $a \leq t \leq b$.

Remark: One could paraphrase Theorem 2 by saying if the local truncation error $\tau_h^{(n)}$ converges with order p , then the one-step methods also converges globally with order p . However, when thinking about the theorem in this way one should not forget the prefactor $(e^{L(t_n-a)} - 1)$ in (14), which grows *exponentially in time*. Thus if we use a one-step method even for a higher order method our results may well deteriorate if $b \gg a$! This point we will pick up in the section on *Stability and implicit methods* below.

Example: Convergence of Forward Euler

As discussed in (10) for Forward Euler has local truncation error $\tau_h^{(n)} = \frac{1}{2}h u''(t_n) + O(h^2)$, such that

$$|\tau_h^{(n)}| \leq \frac{1}{2} \max_{t \in [t_n, t_{n+1}]} |u''(t)| h$$

and using Theorem 2 we conclude that **Forward Euler converges linearly**.

Let us verify this graphically on the test problem (2)

$$\begin{cases} \frac{du(t)}{dt} = \sin[(u+t)^2] & 0 \leq t \leq 4 \\ u(0) = -1, \end{cases}$$

which is contained in the variable `ivp` from before.

Error message

```

UndefVarError: `@L_str` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.

```

```

1 let
2     # Obtain extremely tight solution using DifferentialEquations.jl
3     # Note, that ivp still contains the ODEProblem (2)
4     u_exact = solve(ivp, Tsit5(); reltol=1e-14, abstol=1e-14)
5
6     Ns = [ round(Int, 5 * 10^k) for k in 0:0.5:3 ]
7     errors = Float64[]
8     for N in Ns
9         res_euler = forward_euler(ivp, N)
10        u = res_euler.u
11        t = res_euler.t
12
13        # Compute global errors in each node t_n
14        global_error = [abs(u_exact(t[n]) - u[n]) for n in 1:length(u)]
15
16        # Push the largest error to the array for plotting
17        push!(errors, maximum(global_error))
18    end
19
20    plot(Ns, errors; mark=:o, label="Errors", lw=2, xaxis=:log, yaxis=:log,
21         xlabel=L"N
22         ", ylabel="Largest global error",
23         title="Convergence of Forward Euler")
24
25    # Add line for perfect 1st order convergence:

```

Runge-Kutta methods

Similar to the previous chapters we now ask the question: How can we improve the convergence of a numerical method for an IVP? The answer to this question leads to the major and most widely employed types of methods for solving initial-value problems, the **Runge-Kutta** (RK) methods.

We start with a second-order RK method, namely the midpoint method.

Optional: Midpoint method

The goal of the midpoint method is to construct a second-order method for solving the IVP (1)

$$\begin{cases} \frac{du(t)}{dt} = f(t, u(t)) & a \leq t \leq b \\ u(a) = u_0. \end{cases}$$

We stay within the framework of one-step methods, i.e. we iterate for $n = 0, 1, \dots, N - 1$

$$u^{(n+1)} = u^{(n)} + h \phi_f(u^{(n)}, t_n, h).$$

The ideal method would obtain the exact $u(t_{n+1})$ from the exact $u(t_n)$, implying a local truncation error $\tau_h^{(n)} = 0$. To build such a method we expand

$$\begin{aligned} u(t_{n+1}) &= u(t_n + h) = u(t_n) + h u'(t_n) + \frac{1}{2} h^2 u''(t_n) + O(h^3) \\ &= u(t_n) + h f(t_n, u(t_n)) + \frac{1}{2} h^2 u''(t_n) + O(h^3). \end{aligned} \quad (15)$$

where we used the definition of the IVP to replace u' . If in this equation we keep only the first two terms we recover the Forward Euler method (7). To obtain more accuracy we therefore need to compute or approximate $u''(t_n)$ as well. Using the chain rule we note

$$\frac{d^2 u}{dt^2} = \frac{df}{dt} \stackrel{(*)}{=} \frac{\partial f}{\partial t} + \frac{\partial f}{\partial u} \frac{\partial u}{\partial t} = \frac{\partial f}{\partial t} + \frac{\partial f}{\partial u} f,$$

where notably step $(*)$ is necessary because both f and its argument u depend on t . Inserting this expression into (15) we obtain

$$u(t_{n+1}) = u(t_n + h) = u(t_n) + h \left[f(t_n, u(t_n)) + \frac{h}{2} \frac{\partial f}{\partial t}(t_n, u(t_n)) + \frac{h}{2} \frac{\partial f}{\partial u}(t_n, u(t_n)) f(t_n, u(t_n)) \right] + O(h^3).$$

Since computing and implementing the partial derivatives $\frac{\partial f}{\partial t}$ and $\frac{\partial f}{\partial u}$ can in general become ticky, we will also approximate these further. Expanding f in a multi-dimensional Taylor series we observe

$$f(t_n + \zeta, u(t_n) + \xi) = f(t_n, u(t_n)) + \zeta \frac{\partial f}{\partial t}(t_n, u(t_n)) + \xi \frac{\partial f}{\partial u}(t_n, u(t_n)) + O(\zeta^2 + |\zeta\xi| + \xi^2).$$

With the choice $\zeta = \frac{h}{2}$ and $\xi = \frac{h}{2} f(t_n, u(t_n))$ this expressions becomes equal to lowest order with the term in the square brackets of (16). This leads to

$$u(t_{n+1}) = u(t_n + h) = u(t_n) + h f\left(t_n + \frac{h}{2}, u(t_n) + \frac{h}{2} f(t_n, u(t_n))\right) + O(h^3 + h\zeta^2 + h|\zeta\xi| + h\xi^2).$$

This is the **midpoint method** we mentioned earlier:

Algorithm 2: Midpoint method

Given an initial value problem $\frac{du}{dt} = f(u, t)$, $u(0) = u_0$ with $t \in [a, b]$ and nodes $t_n = a + nh$, $h = \frac{b-a}{N}$ iteratively compute the sequence

$$u^{(n+1)} = u^{(n)} + h \phi_f(u^{(n)}, t_n, h) \quad \forall n = 0, \dots, N-1.$$

with $\phi_f(u^{(n)}, t_n, h) = f\left(t_n + \frac{h}{2}, u^{(n)} + \frac{h}{2} f(t_n, u^{(n)})\right)$

From our derivation the local truncation error for the midpoint method can be identified as

$$\tau_h^{(n)} = \frac{u(t_{n+1}) - u(t_n)}{h} - \phi_f(u(t_n), t_n, h) = \frac{1}{h} O(h^3 + h\zeta^2 + h|\zeta\xi| + h\xi^2) = O(h^2).$$

From Theorem 2 we observe that the **midpoint method** is indeed a **second-order method**.

Runge-Kutta methods, such as the midpoint methods, are what one calls **multi-stage methods**. To make this term clear, let us rewrite equation (7) in two stages by isolating the green part:

$$u^{(n+\frac{1}{2})} := u^{(n)} + \frac{h}{2} f(t_n, u^{(n)})$$

$$u^{(n+1)} = u^{(n)} + h f(t_n + \frac{h}{2}, u^{(n+\frac{1}{2})})$$

Written like this we notice that the **first stage** (in green) performs a Forward Euler step with **half the stepsize**, namely from time t_n to $t_n + \frac{h}{2}$. The **second stage** then performs an Euler-style update over the whole timestep, but employing the slope $u^{(n+\frac{1}{2})}$ from the first stage in the computation of f .

This interpretation also explains why the method is called **midpoint method**.

Employing again the ODESystem to supply the setup of our problem, we obtain an implementation of the midpoint method as:

```
midpoint (generic function with 1 method)
1 function midpoint(ivp, N)
2     a, b = ivp.tspan # Extract time interval [a, b]
3     u0 = ivp.u0      # Initial condition
4     f = ivp.f         # Derivative function f
5     p = ivp.p         # Further parameters (not used in our implementation)
6
7     # Discrete time nodes to consider:
8     h = (b - a) / N
9     t = [a + i * h for i in 0:N]
10
11     # Setup output vector with all elements initialised to u0
12     u = [copy(u0) for i in 0:N]
13
14     # Time integration ... this is what changes over forward_euler
15     u[1] = u0
16     for i in 1:N
17         uhalf = u[i] + h/2 * f(u[i], p, t[i])
18         u[i+1] = u[i] + h * f(uhalf, p, t[i] + h/2)
19     end
20
21     (; t, u)
22 end
```

In comparison with Forward Euler we notice this method to be clearly more accurate for the case of rather small number of timesteps:

Error message

UndefinedVarError: '@L_str' not defined in 'Main.var"workspace#3"'
Suggestion: check for spelling errors or missing imports.

```
1 let
2     N = 9
3     res_euler = forward_euler(ivp, N)
4     res_midpoint = midpoint(ivp, N)
5     plot(sol, label="reference", lw=2, xlabel="t", ylabel=L"u(t)",
6           title=L"\frac{du}{dt} = \sin((t+u)^2)", ylims=(-2, 0.5))
7     plot!(res_euler.t, res_euler.u; label=L"Euler $N=9$", mark=:o, lw=2,
8           ls=:dash)
9     plot!(res_midpoint.t, res_midpoint.u; label=L"Midpoint $N=9$", mark=:o,
10          lw=2, ls=:dash)
11 end
```

👁 Reading hidden

Optional: Higher-order Runge-Kutta methods

The ideas we used for constructing the midpoint method point to a clear path how one could construct even higher-order methods: All we need to do is to introduce further intermediate stages, i.e. half, third or other intermediate timesteps. In this way we can generate additional equations and effectively match the higher-order derivatives in the Taylor expansion (15), which will in turn reduce the local truncation error $\tau_h^{(n)}$. The methods generated in this form are called **Runge-Kutta methods**.

The algebra required to work out the details grows in complexity, so we will not attempt to do this here and only present a general overview. Constructing an s -stage Runge-Kutta methods leads to the set of equations

$$\begin{aligned}v_1 &= h f(t_n, u_n) \\v_2 &= h f(t_n + c_1 h, u_n + a_{11} v_1) \\v_3 &= h f(t_n + c_2 h, u_n + a_{21} v_1 + a_{22} v_2) \\&\vdots \\v_s &= h f\left(t_n + c_{s-1} h, u_n + \sum_{i=1}^{s-1} a_{s-1,i} v_i\right) \\u^{(n+1)} &= u^{(n)} + \sum_{i=1}^s b_i v_i.\end{aligned}$$

where specifying both the number of stages s as well as the constants a_{ij} , b_i and c_i uniquely determines an RK method. Both one-step methods we discussed so far actually match this framework:

- **Forward Euler:** $s = 1$, $b_1 = 1$ and no a_{ij} and no c_i .
- **Midpoint method:** $s = 2$, $b_1 = 0$, $b_2 = 1$, $c_1 = \frac{1}{2}$ and $a_{11} = \frac{1}{2}$.

Additionally we want to specify one additional **fourth-order RK** method, often called **RK4**, which is the most commonly used IVP approach:

Algorithm 3: Fourth-order Runge-Kutta method (RK4)

Given an initial value problem $\frac{du}{dt} = f(u, t)$, $u(0) = u_0$ with $t \in [a, b]$ and nodes $t_n = a + n h$, $h = \frac{b-a}{N}$ iteratively compute the sequence

$$\begin{aligned} v_1 &= h f(t_n, u^{(n)}) \\ v_2 &= h f\left(t_n + \frac{h}{2}, u^{(n)} + \frac{v_1}{2}\right) \\ v_3 &= h f\left(t_n + \frac{h}{2}, u^{(n)} + \frac{v_2}{2}\right) \\ v_4 &= h f(t_n + h, u^{(n)} + v_3) \\ u^{(n+1)} &= u^{(n)} + \frac{v_1}{6} + \frac{v_2}{3} + \frac{v_3}{3} + \frac{v_4}{6} \end{aligned}$$

Let us mention in passing, that our reference method `Tsit5()` is also based on a Runge-Kutta scheme.

An implementation of RK4 is given by:

rk4 (generic function with 1 method)

```
1 function rk4(ivp, N)
2     a, b = ivp.tspan # Extract time interval [a, b]
3     u0 = ivp.u0 # Initial condition
4     f = ivp.f # Derivative function f
5     p = ivp.p # Further parameters (not used in our implementation)
6
7     # Discrete time nodes to consider:
8     h = (b - a) / N
9     t = [a + i * h for i in 0:N]
10
11     # Setup output vector with all elements initialised to u0
12     u = [copy(u0) for i in 0:N]
13
14     # Time integration ... this is what changes over forward_euler
15     u[1] = u0
16     for i in 1:N
17         v1 = h * f(u[i], p, t[i])
18         v2 = h * f(u[i] + v1/2, p, t[i] + h/2)
19         v3 = h * f(u[i] + v2/2, p, t[i] + h/2)
20         v4 = h * f(u[i] + v3, p, t[i] + h)
21
22         u[i+1] = u[i] + (v1/6 + v2/3 + v3/3 + v4/6)
23     end
24
25     (; t, u)
26 end
```

Error message

```
UndefVarError: `Slider` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Stack trace

Here is what happened, the most recent locations are first:

1. macro expansion
from `bonds.jl:125`

2. from `This cell: line 1`

```
1 md"""
```

```
2 Let us compare all methods we saw in this chapter:
```

```

1 md"""
2 Let us compare all methods we saw in this chapter:
3 - `N = ` $(@bind N Slider(5:1:25; show_value=true, default=7))
4 """

```

 Reading hidden

Error message

UndefVarError: `@L\_str` not defined in `Main.var"workspace#3"`
 Suggestion: check for spelling errors or missing imports.

```

1 let
2     res_euler    = forward_euler(ivp, N)
3     res_midpoint = midpoint(ivp, N)
4     res_rk4      = rk4(ivp, N)
5     plot(sol, label="reference", lw=2, ylabel=L"u(t)", xlabel="",
6           title=L"\frac{du}{dt} = \sin((t+u)^2)", ylims=(-2, 0.5), titlefontsize=12)
7     plot!(res_euler.t, res_euler.u; label="Euler", mark=:o, lw=2, ls=:dash,
8            markersize=3)
9     plot!(res_midpoint.t, res_midpoint.u; label="Midpoint", mark=:o, lw=2,
10            ls=:dash, markersize=3)
11    p = plot!(res_rk4.t, res_rk4.u; label="RK4", mark=:o, lw=2, ls=:dash,
12              markersize=3)
13
14    q = plot(; yaxis=:log, ylims=(1e-6, 1), legend=false, ylabel=L"$|u -
15    u_{\text{ref}}|$", xlabel=L"t", title="Error", titlefontsize=10)
16    plot!(q, res_euler.t, abs.(res_euler.u - sol.(res_euler.t) .+ 1e-16);
17           label="Euler", mark=:o, lw=2, markersize=3, ls=:dash, c=2)
18    plot!(q, res_midpoint.t, abs.(res_midpoint.u - sol.(res_midpoint.t) .+ 1e-
19           16);
20           label="Midpoint", mark=:o, lw=2, markersize=3, ls=:dash, c=3)
21    plot!(q, res_rk4.t, abs.(res_rk4.u - sol.(res_rk4.t) .+ 1e-16);
22           label="RK4", mark=:o, lw=2, markersize=3, ls=:dash, c=4)
23
24    plot(p, q; layout=grid(2, 1; heights=[0.75, 0.25]))
25 end

```

 Reading hidden

RK4 converges faster than the other two methods. As the name suggests it is indeed a fourth-order method:

Error message

UndefVarError: `@L\_str` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.

```
1 begin
2     # Obtain extremely tight solution using DifferentialEquations.jl
3     # Note, that ivp still contains the ODEProblem (2)
4     u_exact = solve(ivp, Tsit5(); reltol=1e-14, abstol=1e-14)
5
6     # Function to compute the global error
7     function global_error(t, u, u_exact)
8         errors = [abs(u_exact(t[n]) - u[n]) for n in 1:length(u)]
9         maximum(errors)
10    end
11
12    Ns = [ round(Int, 5 * 10^k) for k in 0:0.5:3 ]
13    errors_euler = Float64[]
14    errors_midpoint = Float64[]
15    errors_rk4 = Float64[]
16    for N in Ns
17        result_euler = forward_euler(ivp, N)
18        push!(errors_euler, global_error(result_euler.t, result_euler.u,
19                                         u_exact))
20
21        result_midpoint = midpoint(ivp, N)
22        push!(errors_midpoint, global_error(result_midpoint.t,
23                                             result_midpoint.u, u_exact))
24
25        res_rk4 = rk4(ivp, N)
26        push!(errors_rk4, global_error(res_rk4.t, res_rk4.u, u_exact))
27    end
28
29    p = plot(; title="Convergence of RK methods", xaxis=:log, yaxis=:log,
30              xlabel=L"n", ylabel="Largest global error", legend=:bottomleft,
31              xlims=(1, 3e4), ylims=(1e-13, 10))
32    plot!(p, Ns, errors_euler; lw=2, mark=:o, c=1, label="Euler")
33    plot!(p, Ns, errors_midpoint; lw=2, mark=:o, c=2, label="Midpoint")
34    plot!(p, Ns, errors_rk4; lw=2, mark=:o, c=3, label="RK4")
35
36    # Add line for perfect 1st order convergence:
37    plot!(Ns, 0.08(first(Ns) ./ Ns).^1, ls=:dash, lw=2, label=L"O(n^{-1})", c=1)
38    plot!(Ns, 0.08(first(Ns) ./ Ns).^2, ls=:dash, lw=2, label=L"O(n^{-2})", c=2)
39    plot!(Ns, 0.08(first(Ns) ./ Ns).^4, ls=:dash, lw=2, label=L"O(n^{-4})", c=3)
40 end
```

On the other hand the four stages of RK4 also have a **disadvantage**: For each timestep four times as many evaluations of the function f are required than Forward Euler. Since this is typically the cost-dominating step we might ask if RK4 should always be employed or if other schemes with less stages (like Midpoint or Euler) are sometimes more appropriate.

To answer this question we take our previous plot and multiply the x-values by the number of stages in order to track the error versus the number of evaluations of the function f . With this we obtain:

Error message

```
UndefVarError: `plot` not defined in `Main.var"workspace#3"`  
Suggestion: check for spelling errors or missing imports.
```

```
1 let  
2     p = plot(; title="Convergence of RK methods", xaxis=:log, yaxis=:log,  
3             xlabel="number of function evaluations",  
4             ylabel="Largest global error", legend=:bottomleft,  
5             xlims=(1, 3e4), ylims=(1e-13, 10))  
6  
7     # Euler has only a single stage  
8     plot!(p, 1Ns, errors_euler; lw=2, mark=:o, c=1, label="Euler")  
9  
10    # Midpoint has two stages  
11    plot!(p, 2Ns, errors_midpoint; lw=2, mark=:o, c=2, label="Midpoint")  
12  
13    # RK4 has four stages  
14    plot!(p, 4Ns, errors_rk4; lw=2, mark=:o, c=3, label="RK4")  
15 end
```

We observe that even though RK4 needs four f evaluations per time step it still provides the best accuracy versus f -evaluations ratio for almost the entire range of evaluations we considered.

Error message

```
Another cell defining logC contains errors.
```

```

1 md"""
2 ## Stability and implicit methods
3
4 Let us consider the innocent looking initial value problem
5 ```math
6 \left\{
7 \begin{aligned}
8 \frac{d u(t)}{d t} &= -10^{\text{logC}} \ \& \ t > 0 \ \backslash\backslash
9 u(0) &= 10.
10 \end{aligned}
11 \right.
12 This model governs for example the decay of a radioactive species with initial
13 concentration $10$ over time. The rate of decay is $10^{\text{logC}}$ $ \approx
14 $(round(10^logC; sigdigits=3)).
15
16 By simple integration we find the exact solution of this problem as:
17 """

```

 Reading hidden

u (generic function with 1 method)

```
1 u(t) = 10 * exp(-10^logC * t)
```

This function rapidly decays to zero as $t \rightarrow \infty$:

Error message

```

UndefVarError: `@L_str` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.

```

```
1 plot(u; xlims=(0, 10), lw=2, title="du/dx = $(round(-10^logC; sigdigits=3))",
titlefontsize=12, label="Reference", xlabel=L"t", ylabel=L"u(t)")
```

Applying our numerical methods from the previous sections, we would expect to recover this behaviour. So let's check this.

We first set up the ODEProblem:

Error message

UndefVarError: `ODEProblem` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.

```
1 decay = let
2   f = (u, p, t) -> -10^logC * u
3   u_0 = 10.0
4   tspan = (0.0, 10.0)
5   decay = ODEProblem(f, u_0, tspan)
6 end
```

... and try it on our previously implemented methods:

Error message

UndefVarError: `Slider` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.

Stack trace

Here is what happened, the most recent locations are first:

1. macro expansion
from `bonds.jl:125`

2. from `This cell: line 1`

```
1 md"""
2 - `logC = `: $(@bind logC Slider(-4:0.2:1; show_value=true, default=0.
3 - `Ndecay = `: $(@bind Ndecay Slider(15:5:50; show_value=true, default=15))
4 """
```

```
1 md"""
2 - `logC = `: $(@bind logC Slider(-4:0.2:1; show_value=true, default=0.8))
3 - `Ndecay = `: $(@bind Ndecay Slider(15:5:50; show_value=true, default=15))
4 """
```

 Reading hidden code

Error message

UndefVarError: `@L\_str` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.

```
1 let
2     h = 10 / Ndecay # Stepsize of our numerical methods
3
4     soln_euler      = forward_euler(decay, Ndecay);
5     soln_midpoint   = midpoint(decay, Ndecay);
6     soln_rk4        = rk4(decay, Ndecay);
7
8     p = plot(u; lw=2, title="du/dx = $(round(-10^logC; sigdigits=3))",
9             titlefontsize=12, label="Reference", xlabel=L"t", ylabel=L"u(t)", xlims=
10            (-0.1, 10.1))
11
12     if h ≥ 2/10^logC # See discussion below to understand
13                     # why this formula appears here
14         ylims!(p, (-1_000, 10_000))
15     end
16
17     plot!(p, soln_euler.t, soln_euler.u; mark=:o, c=2, lw=2, markersize=3,
18           ls=:dash, label="Forward Euler")
19     plot!(p, soln_midpoint.t, soln_midpoint.u; mark=:o, c=3, lw=2, markersize=3,
20           ls=:dash, label="Midpoint")
21     plot!(p, soln_rk4.t, soln_rk4.u; mark=:o, c=4, lw=2, markersize=3, ls=:dash,
22           label="RK4")
23 end
```

We notice for too few nodal points or too large decay constants $10^{\log C}$ our numerical methods **all fail** to recover the decay behaviour. In other words while the exact solution satisfies $\lim_{t \rightarrow \infty} u(t) = 0$, the numerical methods are **all qualitatively wrong**: instead of the reproducing the correct long-time limit numerically, i.e. $\lim_{n \rightarrow \infty} u^{(n)} = 0$, the numerical solution actually grows over time.

Recall that in Theorem 2 we found that the global error satisfies

$$|e^{(n)}| \leq \frac{Ch^p}{L} (e^{L(t_n - a)} - 1) = \frac{Ch^p}{L} (e^{Lhn} - 1).$$

In the limit of infinite time, i.e. $n \rightarrow \infty$, the upper bound on the RHS thus grows exponentially. This bound is in general pessimistic, i.e. not all numerical methods may actually reproduce the exponentially increasing error behaviour, but it clearly shows that unless one demands

additional properties from a method for IVPs, one may fail to reproduce a limiting behaviour like $\lim_{t \rightarrow \infty} u(t) = 0$.

Without going into further details the property we ask for is called **stability**:

Definition: Absolute stability

Given an initial value problem (1) with exact solution $u(t)$ satisfying $\lim_{t \rightarrow \infty} u(t) = u_*$ for any initial condition u_0 . In other words we have a problem where the solution converges to u_* independent of the initial value u_0 .

Then a numerical method is called **absolutely stable** for a fixed stepsize h if

$$\lim_{n \rightarrow \infty} u^{(n)} = u_* \quad \forall u_0.$$

A method is further called **unconditionally absolutely stable** if it is stable for all $h > 0$.

In the case of our decay problem, one can show for example that the forward Euler method is absolutely stable if

$$h < \frac{2}{10^{\log C}}$$

— which explains the condition we chose to switch the axis limits in the plot above.

In particular we also observe that none of our methods is unconditionally absolutely stable.

Backward Euler

The construction and implementation of unconditionally absolutely stable methods is in general more involved than the methods we considered so far. We will therefore only construct a single approach here.

For this we return to the derivation of the Forward Euler method. Recall that in order to solve the IVP (1)

$$\begin{cases} \frac{du(t)}{dt} = f(t, u(t)) & a \leq t \leq b \\ u(a) = u_0, \end{cases}$$

we introduced a time discretisation into N subintervals $[t_n, t_{n+1}]$, such that in each interval we needed to solve the problem

$$\frac{du(t_n)}{dt} = f(t_n, u(t_n)). \quad (20)$$

Instead of employing the forward finite differences formula, leading to the Forward Euler method (7), we now instead employ the backward finite differences, leading to

$$\frac{du(t_{n+1})}{dt} \approx D_h^- u(t_{n+1}) = \frac{1}{h} \left(u(t_{n+1}) - u(t_{n+1} - h) \right) = \frac{1}{h} \left(u(t_{n+1}) - u(t_n) \right).$$

Following the same idea as before, i.e. to employ for $u(t_{n+1})$ the approximation $u^{(n+1)}$ of the $n+1$ -st time step and instead of $u(t_n)$ employ $u^{(n)}$ we obtain from (20) and (21):

$$\frac{u^{(n+1)} - u^{(n)}}{h} = f(t_{n+1}, u^{(n+1)}), \quad \forall n = 0, \dots, N-1, \quad (22)$$

which is the **Backwards Euler method**. Notice, that in contrast to the case of Forward Euler we cannot immediately deduce an explicit update formula like (7) from this expression, since the dependency on $u^{(n+1)}$ is both on the LHS as well as in f . This is why such methods are called **implicit methods** as $u^{(n+1)}$ is only implicitly defined.

To obtain $u^{(n+1)}$ from $u^{(n)}$ one needs to solve (22) iteratively: For this we introduce the map

$$x = g^{(n)}(x) = u^{(n)} + h f(t_{n+1}, x) \quad (23)$$

and notice that its fixed-point $x_* = g^{(n)}(x_*)$ is exactly $u^{(n+1)}$. For each time step n we thus need to solve a fixed-point problem using one of the methods we described in chapter 4.

In summary our algorithm becomes:

Algorithm 4: Backward Euler method

Given an initial value problem $\frac{du}{dt} = f(u, t)$, $u(0) = u_0$ with $t \in [a, b]$ and nodes $t_n = a + nh$, $h = \frac{b-a}{N}$ we iterate for $n = 1, \dots, N$:

- Find a fixed-point x_* of the map $g^{(n)}(x) = u^{(n)} + h f(t_{n+1}, x)$, e.g. using the Newton method.
- Set $u^{(n+1)} = x_*$

An implementation of Backward Euler employing Newton's method to solve the fixed-point problem is given below. The implementation of Newton's method is repeated in the appendix.

backward_euler (generic function with 1 method)

```
1 function backward_euler(ivp, N; tol=1e-10)
2     a, b = ivp.tspan # Extract time interval [a, b]
3     u0 = ivp.u0 # Initial condition
4     f = ivp.f # Derivative function f
5     p = ivp.p # Further parameters (not used in our implementation)
6
7     # Discrete time nodes to consider:
8     h = (b - a) / N
9     t = [a + i * h for i in 0:N]
10
11     # Setup output vector with all elements initialised to u0
12     u = [copy(u0) for i in 0:N]
13
14     # Time integration
15     u[1] = u0
16     for i in 1:N
17         g(x) = u[i] + h * f(x, p, t[i+1])
18         dg(x) = ForwardDiff.derivative(g, x)
19         u[i+1] = fixed_point_newton(g, dg, u[i]; tol).fixed_point
20     end
21
22     (; t, u)
23 end
```

Error message

UndefVarError: `Slider` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.

Stack trace

Here is what happened, the most recent locations are first:

1. macro expansion
from [bonds.jl:125

2. from [This cell: line 1

```
1 md"""
2 - `Nbww = ` $(@bind Nbww Slider(15:5:40; show_value=true, default=15) )
```

```
1 md"""
2 - `Nbww = ` $(@bind Nbww Slider(15:5:40; show_value=true, default=15) )
```

Error message

```
UndefVarError: `@L_str` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

```
1 let
2     soln_euler      = forward_euler(decay, Nbw)
3     soln_bw_euler = backward_euler(decay, Nbw)
4
5     p = plot(u; lw=2, title="du/dx = $(round(-10^logC; sigdigits=3))",
6             titlefontsize=12, label="Reference", xlabel=L"t", ylabel=L"u(t)", xlims=
7             (-0.1, 10.1), ylims=(-1, 25))
8
9     plot!(p, soln_euler.t, soln_euler.u; mark=:o, c=2, lw=2, markersize=3,
10          ls=:dash, label="Forward Euler")
11     plot!(p, soln_bw_euler.t, soln_bw_euler.u; mark=:o, c=3, lw=2, markersize=3,
12          ls=:dash, label="Backward Euler")
13 end
```

While Forward Euler stays absolutely stable only for large values of N_{bw} (respectively small values of h), Backward Euler stays stable no matter what value of N_{bw} is chosen.

Let us conclude by mentioning that similar to Forward Euler, **Backward Euler** is also only a **first order method**. Higher-order implicit methods can also be constructed. E.g. the Runge-Kutta family of methods can be extended to the implicit setting as well. An example is the Crank–Nicolson method, a second-order implicit Runge-Kutta method.

In standard libraries, such as DifferentialEquations.jl a zoo of ODE methods is typically available.

Optional: Higher-order differential equations

Consider the setting of a mass m hanging on a spring from the ceiling:

Error message

```
UndefVarError: `TODO` not defined in `Main.var"workspace#3"`  
Suggestion: check for spelling errors or missing imports.
```

Stack trace

Here is what happened, the most recent locations are first:

1. from | This cell: line 1

```
1 TODO("Image")
```

```
1 TODO("Image")
```

In its rest position shown above, gravity is exactly cancelled by the upward force exerted by the spring, such that we can ignore gravity for our discussion. Now displacing the mass by a length x leads to a restoring force $-kx$ where k is the spring constant of the spring. By Newton's law this accelerates the mass by $-k/mx$. Employing $m = 1$ for simplicity this leads to the **second-order ODE**

$$\begin{cases} \frac{d^2 x(t)}{dt^2} = -k x(t) & t > 0 \\ x(0) = 0, \end{cases}$$

which is usually referred to as the **simple harmonic oscillator model**, abbreviated HO. To solve this problem numerically we first introduce the velocity function $v(t)$ and rewrite it as a system of first-order ODEs:

$$\begin{cases} \frac{dx(t)}{dt} = v(t) & t > 0 \\ \frac{dv(t)}{dt} = -k x(t) & t > 0 \\ x(0) = 0 \\ v(0) = 0.5, \end{cases} \quad (17)$$

where we choose to start at the rest position, but with an initial velocity of **0.5**. Collecting the position and velocity into a single vector-valued variable, i.e.

$$\mathbf{u}(t) = \begin{pmatrix} u_1(t) \\ u_2(t) \end{pmatrix} = \begin{pmatrix} x(t) \\ v(t) \end{pmatrix}, \quad \frac{d\mathbf{u}(t)}{dt} = \begin{pmatrix} \frac{d u_1(t)}{dt} \\ \frac{d u_2(t)}{dt} \end{pmatrix} = \begin{pmatrix} \frac{d x(t)}{dt} \\ \frac{d v(t)}{dt} \end{pmatrix}, \quad \mathbf{u}_0 = \begin{pmatrix} 0 \\ 0.5 \end{pmatrix}$$

and introducing the function $\mathbf{f}: \mathbb{R}^2 \times \mathbb{R} \rightarrow \mathbb{R}^2$ with

$$\mathbf{f}(\mathbf{u}, t) = \begin{pmatrix} u_2(t) \\ -k u_1(t) \end{pmatrix}$$

we can rewrite this as

$$\begin{cases} \frac{d\mathbf{u}(t)}{dt} = \mathbf{f}(\mathbf{u}, t) & t > 0 \\ \mathbf{u}(0) = \mathbf{u}_0. \end{cases} \quad (18)$$

Notice, that this has exactly the same structure as (1), just all quantities are vector valued.

All algorithms we discussed in this chapter so far can be extended to this setting. For example Algorithm 1 (Forward Euler) simply becomes:

Algorithm 1a: Forward Euler for vector-valued problems

Given an initial value problem $\frac{d\mathbf{u}}{dt} = \mathbf{f}(t, \mathbf{u})$, $\mathbf{u}(0) = \mathbf{u}_0$ with $t \in [a, b]$ and nodes $t_n = a + n h$, $h = \frac{b-a}{N}$ iteratively compute the sequence

$$\mathbf{u}^{(n+1)} = \mathbf{u}^{(n)} + h \mathbf{f}(t_n, \mathbf{u}^{(n)}) \quad \forall n = 0, \dots, N-1. \quad (19)$$

which is identical to Algorithm 1, just with all quantities replaced by their vector-valued analogues. In fact even the implementations of `forward_euler` can just be used without changing a single line of code as we will demonstrate now.

First we setup the HO model as an `ODEProblem`:

```
k = 2
1 k = 2 # Force constant
```

Error message

UndefVarError: `ODEProblem` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.

Stack trace

Here is what happened, the most recent locations are first:

1. from `This cell: line 13`

```
12  
13 ho = ODEProblem(f, u₀, tspan)  
14 end
```

```
1 ho = let  
2   # To setup f we assume u is a vector of size 2 and return a vector of size 2  
3   f = (u, p, t) -> [ u[2];  
4                     -k * u[1]]  
5  
6   # As the initial value again we supply a vector of size 2  
7   u₀ = [0.0;  
8         1.0]  
9  
10  # And we are interested in the behaviour from 0 to tend (defined below)  
11  tspan = (0.0, 10.0)  
12  
13  ho = ODEProblem(f, u₀, tspan)  
14 end
```

Then running the dynamics just takes a call to `forward_euler` as before:

Error message

Another cell defining `ho` contains errors.

```
1 ho_euler = forward_euler(ho, 50);
```

Error message

```
UndefVarError: `@L_str` not defined in `Main.var"workspace#3"`  
Suggestion: check for spelling errors or missing imports.
```

```
1 let  
2   x = [u[1] for u in ho_euler.u] # Extract the particle position  
3   plot(ho_euler.t, x, mark=:o, c=2, lw=2, xlabel=L"t", ylabel=L"x(t)",  
         label="", title=L"\frac{d^2 x}{d t^2} = -k x", titlefontsize=12,  
         markersize=3, ls=:dash)  
4 end
```

In fact the midpoint and rk4 implementations we provide are similarly generic with respect to being used with scalar-valued or vector-valued u . Applying them all we obtain:

Error message

```
UndefVarError: `Slider` not defined in `Main.var"workspace#3"`  
Suggestion: check for spelling errors or missing imports.
```

Stack trace

Here is what happened, the most recent locations are first:

1. macro expansion
from `bonds.jl:125`
2. from `This cell: line 1`

```
1 md""  
2 - `Nho = ` $(@bind Nho Slider(5:5:100; show_value=true, default=50) )
```

```
1 md""  
2 - `Nho = ` $(@bind Nho Slider(5:5:100; show_value=true, default=50) )  
3 ""
```

 Reading hidden code

Error message

UndefVarError: `@L\_str` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.

```
1 let
2     # Find a reference solution using Tsit5():
3     reference = solve(ho, Tsit5(); abstol=1e-14, reltol=1e-14);
4
5     # Solve the problem using all our implemented RK methods
6     soln_euler = forward_euler(ho, Nho)
7     soln_midpoint = midpoint(ho, Nho)
8     soln_rk4 = rk4(ho, Nho)
9
10    # Function to compute the pointwise error in the positions
11    function compute_error(solution)
12        t = solution.t
13        u = solution.u
14        [abs(reference(t[n])[1] - u[n][1]) .+ 1e-16 for n in 1:length(u)]
15    end
16
17    # HO plot x versus t
18    p = plot(t -> reference(t)[1]; # plot only position
19           lw=2, label="reference", ylabel=L"x(t)", xlims=ho.tspan,
20           title=L"\frac{d^2 x}{d t^2} = -k x", titlefontsize=12, ylims=(-3,
21           3),
22           legend=:bottomleft)
23    plot!(p, soln_euler.t, [u[1] for u in soln_euler.u];
24          label="Forward Euler", lw=2, mark=:o, ls=:dash, c=2, markersize=3)
25    plot!(p, soln_midpoint.t, [u[1] for u in soln_midpoint.u];
26          label="Midpoint", lw=2, mark=:o, ls=:dash, c=3, markersize=3)
27    plot!(p, soln_rk4.t, [u[1] for u in soln_rk4.u];
28          label="RK4", lw=2, mark=:o, ls=:dash, c=4, markersize=3)
29
30    # Total energy plots
31    q = plot(; legend=false, xlabel=L"t", title="Error", titlefontsize=10,
32             yaxis=:log, ylims=(1e-5, 10), xlims=ho.tspan)
33
34    plot!(q, soln_euler.t, compute_error(soln_euler); label="Euler", lw=2,
35          ls=:dash, c=2)
36    plot!(q, soln_midpoint.t, compute_error(soln_midpoint); label="Midpoint",
37          lw=2, ls=:dash, c=3)
38    plot!(q, soln_rk4.t, compute_error(soln_rk4); label="RK4", lw=2, ls=:dash,
39          c=4)
```

```
37 plot(p, q; layout=grid(2, 1; heights=[0.75, 0.25]))
```

Appendix

.....

fixed_point_newton (generic function with 1 method)

```
1 function fixed_point_newton(g, dg, xstart; maxiter=40, tol=1e-6)
2     f(x) = g(x) - x
3     df(x) = dg(x) - 1
4     newton(f, df, xstart; maxiter, tol)
5 end
```

newton (generic function with 1 method)

```
1 function newton(f, df, xstart; maxiter=100, tol=1e-6)
2     # f: Function of which we seek the roots
3     # df: Function, which evaluates its derivatives
4     # xstart: Start of the iterations
5     # maxiter: Maximal number of iterations
6     # tol: Convergence tolerance
7
8     # Define the fixed-point function g_Newton using f and df
9     g_Newton(x) = x - f(x) / df(x)
10
11     # Solve for its fixed point:
12     fixed_point_iterations(g_Newton, xstart; tol, maxiter)
13 end
```

fixed_point_iterations (generic function with 1 method)

```
1 function fixed_point_iterations(g, xstart; tol=1e-6, maxiter=100)
2     # g:      Fixed-point function
3     # xstart: Initial guess
4     # tol:    Tolerance
5
6     history_x = [float(xstart)]
7     history_r = empty(history_x)
8
9     r(k) = Inf # For initial pass in while loop
10    k = 0
11    while k < maxiter && abs(r(k)) ≥ tol
12        x(k) = last(history_x) # Take out the most recent point from the history
13        x(k+1) = g(x(k))
14        r(k) = x(k+1) - x(k)
15        push!(history_r, r(k))
16
17        k = k + 1 # Update k
18        push!(history_x, x(k+1)) # Push next point to the history
19    end
20
21    # Return results as a named tuple
22    (; fixed_point=last(history_x), residual=r(k), n_iter=k, history_x,
23    history_r)
```

Error message

```
UndefVarError: `@htl` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Stack trace

Here is what happened, the most recent locations are first:

1. from [:0
2. #macroexpand#74
from [expr.jl:125
3. macroexpand
from [expr.jl:123

```
1 let
```

```
2 RobustLocalResource("https://teaching.matmat.org/numerical-
analysis/sidebar.md", "sidebar.md")
3 Sidebar(toc, ypos) = @html("""<aside class="plutoui-toc aside indent"
4 style='top:$(ypos)px; max-height: calc(100vh - $(ypos)px - 55px);'
>${toc}</aside>""")
5 Sidebar(Markdown.parse(read("sidebar.md", String)), 570)
6 end
```

 Reading hidden
code